

Unseen Data 2025 Starting Kit

Hi, thanks for participating in the 5th NAS Unseen-Data Competition!

To find out more information, including dates and rules, please visit our website: <https://www.nascompetition.com>.

Contents

The starting kit contains the following:

- `evaluation/` : These are copies of scripts that will be used to evaluate your submission on our servers.
 - `main.py` : The main competition pipeline. This will load each dataset, pass it through your pipeline, and then produce test predictions
 - `score.py` : The scoring script, which will compare the test predictions from `main.py` and compare it against the true labels.
- `submission_template/` : This contains everything you need to implement to create a valid submission. See the included README within for more details
- `submission_example/` : Here's an example submission we made, for reference
- `Makefile` : Some scripts that will let you build and test your submission in a copy of our server evaluation pipeline, more details on this in the "Testing your Submission" section

Datasets

The final datasets your work will be evaluated on will be kept hidden until the end of the competition. However, below we have provided links to datasets created for previous iterations of the competition. Please create a `datasets` directory in the main folder of the starting kit and add datasets, either from our collection or elsewhere.

Our pipeline and DataLoaders are expecting each dataset to be contained in its own folder with six NumPy files for the training, validation, and testing data, split between images and labels. Furthermore, a `metadata` file is expected containing the input shape, codename, benchmark, and number of classes. See the datasets we created (linked below), for the appropriate structure.

Important Note: We have updated the 2026 starter kit so that the time limit is defined per dataset. The default is defined in the code as `30 minutes`. To test and modify this, please add `time_limit` as a variable in the metadata file

Example metadata for AddNIST with a time limit of 1 hour:

```
{"num_classes": 20, "input_shape": [50000, 3, 28, 28], "codename": "Adaline", "benchmark": 89.850, "time_limit": 1.0}
```

- AddNIST: <https://doi.org/10.25405/data.ncl.24574354.v1>
- Language: <https://doi.org/10.25405/data.ncl.24574729.v1>
- MultNIST: <https://doi.org/10.25405/data.ncl.24574678.v1>
- CIFARTile: <https://doi.org/10.25405/data.ncl.24551539.v1>
- Gutenberg: <https://doi.org/10.25405/data.ncl.24574753.v1>
- GeoClassing: <https://doi.org/10.25405/data.ncl.24050256.v3>
- Chesseract: <https://doi.org/10.25405/data.ncl.24118743.v2>
- Sudoku: <https://doi.org/10.25405/data.ncl.26976121.v1>
- Voxel: <https://doi.org/10.25405/data.ncl.26970223.v1>
- Myofibre: <https://doi.org/10.25405/data.ncl.26969998.v1>
- GameOfLife: <https://doi.org/10.25405/data.ncl.30000835>
- Cryptic: <https://doi.org/10.7488/ds/8054>
- Windspeed: <https://doi.org/10.7488/ds/8053>

Writing Your Submission

In this competition, you will be asked to produce three components:

1. A `DataProcessor`, that takes in raw numpy arrays comprising the train/valid/splits of the dataset and creates train/valid/test PyTorch dataloaders. These can perform whatever preprocessing or augmentation that you might want/
2. A NAS algorithm, that takes in the dataloaders and produces some optimal PyTorch model
3. A `Trainer`, that trains that optimal model over the train dataloader

In general, the following pipeline occurs for each dataset:

1. Raw Dataset -> `DataProcessor` -> Train, Valid, and Test dataloaders
2. Train Dataloader + Valid Dataloaders -> NAS -> Model
3. Model + Train Dataloader + Valid Dataloaders -> `TRAINER.train` -> Fully-trained model
4. Fully-trained model + Test Dataloader -> `Trainer.predict` -> Predictions

See `submission_template/README.md` for specifics about how to write these, and `'submission_example'` for an example valid submission

Runtime

Inside the `evaluation/main.py` file we create a clock that is passed through to the three components listed above. This can use to check the time remaining. The time limit is set with inside `main.py` by the `TIME_LIMIT` constant, by default this is set to 12 hours. When we test your code in phase two, you will only be given **one** hour as we are just testing the code works. We will use the **same** submission from phase 2 for the final run in phase 3. This final run which will be given an unknown amount of runtime. It is your job to use the clock to manage the amount of time your code has and to adapt to the amount of time given.

Note. this year we have also added code that will terminate a submission once it has exceeded the `TIME_LIMIT` , this is also in `evaluation/main.py` . This countdown is seperate to the clock. We will use our own versions of `main.py` and `score.py` and any attempts to extend your time limit will result in disqualification.

Testing Your Submission

The included Makefile will let you test your submission via the same testing scripts as our servers use. If the Makefile works, then you can be fairly confident your submission will work on our machines. However, you should still be careful about things like package imports, because trying to import something that doesn't exist in our environment will break your submission.

To test your submission from start-to-finish, run:

```
make submission=$SUBMISSION_DIRECTORY all
```

For example, to run the example submission:

```
make submission=submission_example all
```

Submitting

To bundle your submission, run:

```
make submission=$SUBMISSION_DIRECTORY zip
```

Then submit the zip file by sending it to us via email at nas-competition-contact@newcastle.ac.uk.